



TECNOLOGÍAS PARA EL DESARROLLO DE APLICACIONES WEB

Unidad I. Aspectos básicos del desarrollo de aplicaciones Web

Objetivo

Identifica los elementos principales en el desarrollo de aplicaciones Web así como las buenas prácticas a partir de la arquitectura y los estándares de la Web.

Patricia Escamilla Miranda

Unidad I. Aspectos básicos del desarrollo de aplicaciones Web

1.1 Evolución histórica de Internet y la WWW

[Ver presentación](#)

A continuación, se enlistan algunos términos utilizados con frecuencia y que tenemos que comprender:

Página Web: es un documento digital capaz de incluir textos, imágenes, audios, videos, etc., a la cual se puede acceder a través de un navegador Web.

Navegador Web: es un programa informático que permite al usuario ingresar a las páginas Web que desee. Teniendo la dirección URL (Uniform Resource Locator) o a través de un buscador.

Hipertexto: es una estructura que organiza la información de forma no lineal, lo que permite saltar de un punto a otro en un texto, a través de los enlaces.

Hipervínculo: es una conexión unidireccional entre dos espacios virtuales en el mundo digital. Dicha conexión es creada a través de tags.

Internet compuesto por el vocablo inter (entre) y net (proveniente de network que quiere decir red electrónica): red de computadoras que se encuentran interconectadas a nivel mundial para compartir información.

World Wide Web (red de alcance mundial), es un complejo sistema de hipertextos e hipermedios interconectados entre sí y a los que puedes tener acceso mediante una conexión a internet y con un software especializado.

1.2 La internet y la WWW

[Ver video](#)

Tecnologías para el desarrollo de aplicaciones Web

Para lograr el impresionante flujo de datos que interconecta el planeta, la WWW hace uso de los llamados estándares web, sin los cuales este movimiento de información sería imposible.

Esos estándares básicos son:

- **HTTP** Protocolo de Transferencia de Hipertexto, estándar que especifica cómo se comunican los elementos involucrados (navegador y servidor).
- **URI** Identificador de Recurso Uniforme, el cual básicamente es un sistema que permite referenciar recursos en la Web como páginas web y darles un valor único que impida confusiones en su ubicación.

Partes de la URL



Ilustración 1. Partes de la URL.

Las URL pueden ser **absolutas** (son aquellas que incluyen toda la información necesaria para acceder al recurso) o **relativas** (son aquellas que solo declaran la parte de la ruta que el navegador tiene que agregar a la URL actual para acceder al recurso y se puede determinar una ruta hacia abajo y hacia arriba de la jerarquía usando los caracteres `../` al comienzo de la ruta). Por ejemplo, si tenemos un hipervínculo que hace referencia a una imagen dentro del directorio imágenes, podemos crear el enlace con la URL `http://www.ejemplo.com/imagenes/miimagen.png`, pero también tenemos la opción de declararla como “`imagenes/miimagen.png`” y el navegador se encargará de agregar a esta ruta la URL actual y descargar la imagen.

Dominio: es simplemente un nombre personalizado con una extensión que determina el propósito del sitio web (`.com`, `.org`, `.net`, etc.).

Alojamiento web (web hosting): es un servicio de alquiler de un servidor configurado y listo para almacenar, y operar uno o múltiples sitios web (File Manager, FTP Accounts, phpMyAdmin, Email Accounts, entre otros).

1.2.1 Arquitectura Cliente/Servidor

[Ver presentación](#)

1.2.2 Protocolo TCP/IP

- Es un conjunto de protocolos que sirven para comunicar dos computadoras.
- Es el encargado de manejar los errores en la transmisión, administrar el enrutamiento y entrega de los datos.

Topología de Red

Describe la disposición de los sistemas en una computadora red.

Define cómo las computadoras, o nodos, dentro de la red están dispuestos y conectados entre sí. Las topologías pueden ser físicas y lógicas.

Topología física: es la manera en cómo están conectados los elementos de la red, las categorías son bus, anillo, estrella y malla:

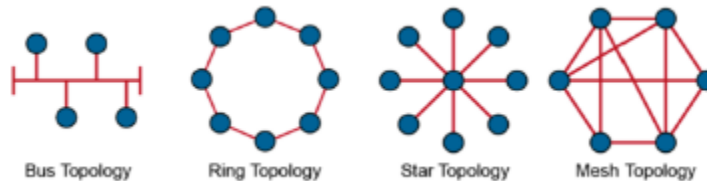


Ilustración 2. Tipos de Topologías.

Topología lógica: es la manera de cómo se comunican los datos a través de la red.

Protocolos

Es el conjunto de reglas que gobiernan el intercambio de datos entre dos entidades (programas de aplicación de los usuarios, los sistemas de gestión de bases de datos, gestores de correo).

Se encuentran disponibles muchos protocolos que rigen la comunicación humana correcta. Una vez que se acuerda un método de comunicación (cara a cara, teléfono, carta, fotografía), los protocolos implementados deben contemplar los siguientes requisitos:

- Un emisor y un receptor identificados
- Idioma y gramática común

Tecnologías para el desarrollo de aplicaciones Web

- Velocidad y temporización de la entrega
- Requisitos de confirmación o acuse de recibo

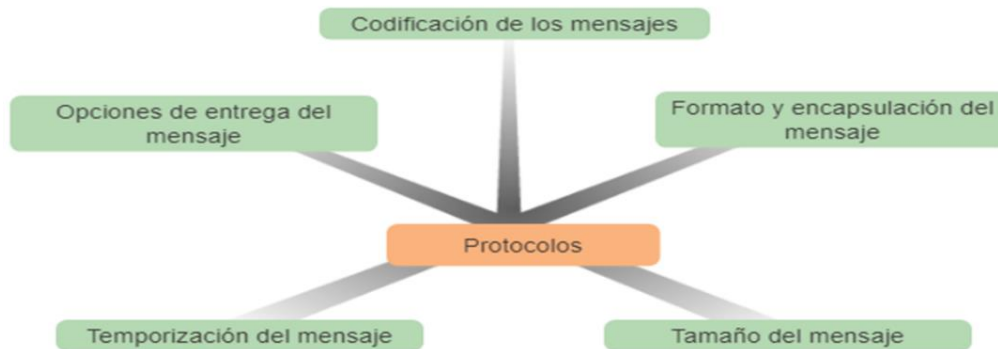


Ilustración 3. Funciones de los protocolos.

Codificación del mensaje: Proceso mediante el cual la información se convierte en otra forma aceptable para la transmisión.

Formato: El formato y el contenido de una trama están determinados por el tipo de mensaje que se envía y el canal que se utiliza para enviarlo.

Tamaño del mensaje: cuando se envía un mensaje largo de un host a otro a través de una red, es necesario dividirlo en partes más pequeñas.

Temporización: para determinar cuándo hablar, la velocidad con la que lo harán y cuánto tiempo deben esperar una respuesta.

El método de acceso: determina en qué momento alguien puede enviar un mensaje.

Tipos básicos de Modelos de Networking

Existen dos tipos básicos: **modelos de protocolo** y **modelos de referencia**.

Un **modelo de protocolo** es un modelo que describe las funciones que se producen en cada capa y de los protocolos dentro del conjunto TCP/IP.

El **modelo OSI (de interconexión de sistema abierto)** es un **modelo de referencia** de internetwork, se utiliza para el diseño de redes de datos, especificaciones de funcionamiento y resolución de problemas.

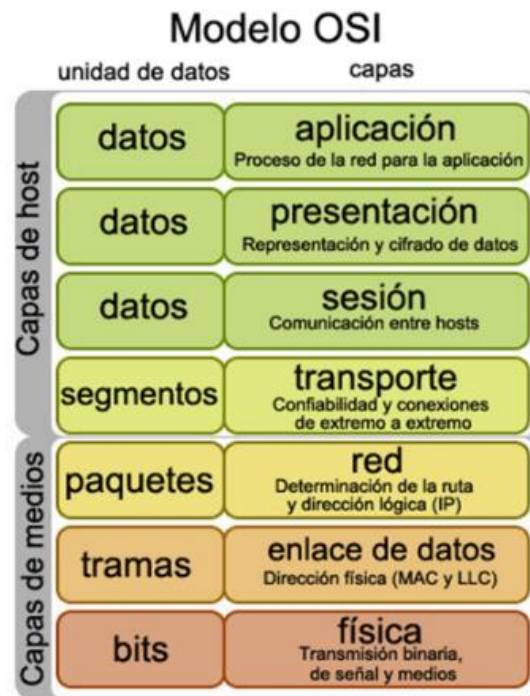


Ilustración 4. Ejemplo del modelo de referencia (Modelo OSI)

Para mayor referencia de cómo trabaja el modelo OSI y TCP/IP, te invito a ver el siguiente [vídeo](#).

El conjunto de protocolos TCP/IP considera las siguientes capas:

- **Capa de aplicación:** es la más cercana al usuario y la que define los protocolos que usarán las aplicaciones que proporcionan servicios al usuario
- **Capa de transporte:** establece los canales básicos que utilizarán las aplicaciones para la transferencia de datos extremo a extremo (o entre dos puntos). Esta capa puede incluir mecanismos de seguridad y hay protocolos que se encargan de la segmentación de paquetes, del control de errores y el control de flujo de estos.
- **Capa Internet:** se ocupa de la estructura del paquete de datos básico que circula por la red y de cómo enviarlos a través de la red. También identifica el host en la red mediante una dirección IP y el direccionamiento de estos datagramas dando saltos entre enrutadores hasta llegar al destino.

- **Capa acceso a la red:** define el acceso físico de los equipos conectados a la red y los protocolos que en él intervienen, la topología física y como se mueven los paquetes entre las interfaces de la capa de Internet.

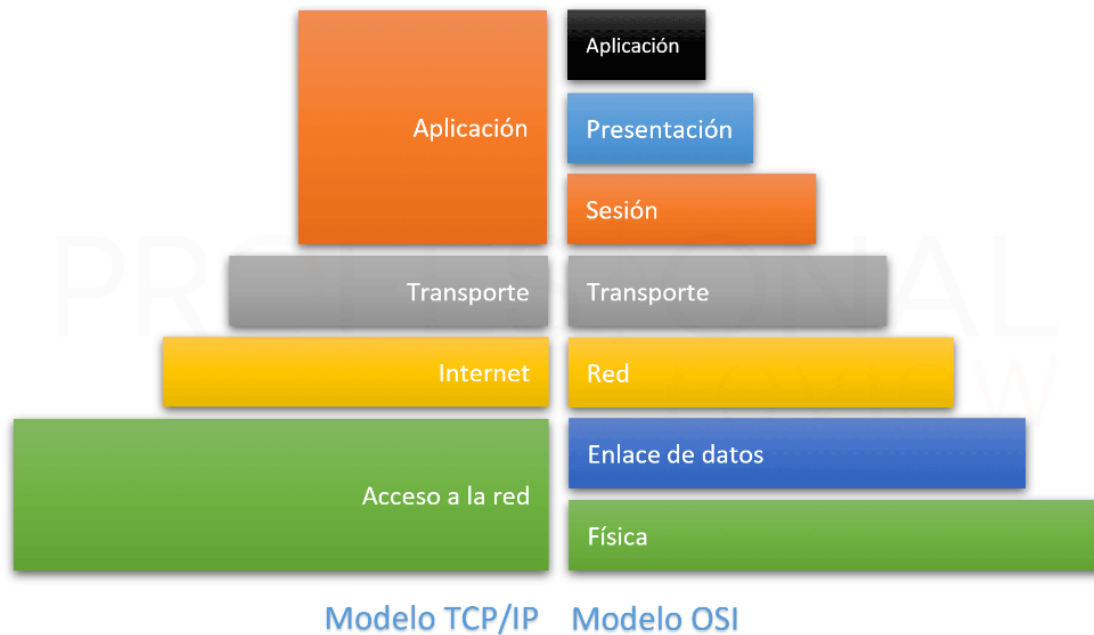


Ilustración 5. Conjunto de protocolos TCP/IP.

TCP (Transmission Control Protocol)

Es uno de los principales protocolos de la capa de transporte del modelo TCP/IP, encargado de establecer y permitir la conexión entre dos hosts y asegurar el intercambio de datos entre ellos. Cuando se proporcionan los datos al protocolo IP, los agrupa en datagramas (es un paquete de datos *que* constituye el mínimo bloque de información en una red de conmutación por *datagramas*) IP y le añade su propio encabezado y cabecera para asegurar su correcto transporte.

IP Internet Protocol

El **protocolo IP** es parte de la capa de Internet del conjunto de protocolos TCP/IP, encargado de proporcionar una dirección IP a todos los equipos conectados a una red, ya sean hosts cliente o servidores. Es uno de los protocolos de Internet más importantes ya que permite el desarrollo y transporte de datagramas de IP (paquetes de datos), aunque sin garantizar su "entrega" ni

orden de los paquetes (ya que es un protocolo no orientado a conexión, es decir, que el intercambio de datos se puede hacer sin un acuerdo previo entre cliente y servidor o emisor y receptor). En realidad, el protocolo IP procesa datagramas de IP de manera independiente al definir su representación, ruta y envío.

El protocolo IP determina el destinatario del mensaje mediante 3 campos:

- **el campo de dirección IP:** Dirección del equipo;
- **el campo de máscara de subred:** una máscara de subred le permite al protocolo IP establecer la parte de la dirección IP que se relaciona con la red;
- **el campo de pasarela predeterminada:** le permite al protocolo de Internet saber a qué equipo enviar un datagrama, si el equipo de destino no se encuentra en la red de área local.



Ilustración 6. Protocolos con los que trabaja el Modelo TCP/IP.

Las aplicaciones TCP/IP tienden a operar de acuerdo con el paradigma **cliente/servidor**. Significa que, en estas aplicaciones, los servidores (también llamados servicios y demonios, según el idioma del sistema operativo subyacente) se ejecutan mediante:

- Esperar a que lleguen las solicitudes de los programas cliente y luego,
- Procesar esas solicitudes

Los programas cliente pueden ser aplicaciones utilizadas por seres humanos, o pueden ser servidores que necesitan hacer sus propias solicitudes que solo pueden ser cumplidas por otros servidores. La mayoría de las veces, el cliente y el servidor se ejecutan en máquinas separadas y se comunican con una conexión a través de una red.

¿Cómo se comunican entre sí los clientes y servidores TCP/IP?

Para hablar con los servidores, los programas cliente TCP/IP abren un socket, que es simplemente una conexión TCP entre la máquina cliente y la máquina servidor. Los servidores escuchan solicitudes de conexión que llegan a través de puertos específicos.

Un puerto es una referencia numérica dentro una solicitud que indica qué programa de servidor es su destinatario.

Los servidores escuchan solicitudes en números de puerto conocidos. Por ejemplo, los servidores SMTP escuchan el puerto 25 y los **servidores web** escuchan el puerto **80**.

1.2.3 Protocolo HTTP

Para comenzar el tema del protocolo HTTP, hay que revisar algunas definiciones, así como, recordar datos revisados anteriormente:

¿Qué es una 'aplicación web'?

Es una aplicación cliente/servidor que utiliza un navegador web como su programa cliente y realiza un servicio interactivo al conectarse con servidores a través de Internet (o Intranet). Un sitio web simplemente entrega contenido de archivos estáticos. Una aplicación Web presenta contenido personalizado dinámicamente basado en parámetros de solicitud, comportamientos de usuario rastreados y consideraciones de seguridad.

Recordando algunos puntos importantes a finales de 1990, Berners-Lee había implementado un servidor y un navegador con línea de comandos utilizando la versión inicial del Protocolo de transferencia de hipertexto (HTTP) que él diseñó para la World Wide Web.

Había tres componentes básicos ideados por Tim Berners-Lee que comprenden la esencia de la tecnología Web:

1. Un lenguaje de marcado para formatear documentos de hipertexto y permitía la referencia cruzada de documentos a través de hipervínculos (HTML)
2. Un esquema de notación uniforme (se le denomina Identificador Uniforme de Recursos - URI) para direccionar los recursos accesibles a través de la red. Conocido como URL.
3. Un protocolo para transportar mensajes a través de la red. Base fundamental de la World Wide Web.

HTTP es un **protocolo de nivel de aplicación** en el conjunto de protocolos TCP/IP, que utiliza TCP como el protocolo de capa de transporte subyacente para transmitir mensajes. Lo fundamental que vale la pena saber sobre el protocolo HTTP y la estructura de los mensajes es que:

- *El protocolo HTTP utiliza el paradigma de solicitud/respuesta, lo que significa que un programa HTTP cliente envía un mensaje de solicitud HTTP a un servidor HTTP, que devuelve un mensaje de respuesta HTTP.*
- *La estructura de los mensajes de solicitud y respuesta es similar a la de los mensajes de correo electrónico; consisten en un grupo de líneas que contienen encabezados de mensajes, seguidos de una línea en blanco, seguida de un cuerpo de mensaje.*

HTTP es un protocolo sin estado, lo que significa que no tiene soporte explícito para la noción de estado. Una transacción HTTP consiste en una sola solicitud de un cliente a un servidor, seguido de una sola respuesta del servidor al cliente.

En la práctica, los servidores y los navegadores rara vez se comunican directamente; hay uno o más proxies (son programas que actúan como servidores y clientes, realizando solicitudes a servidores web en nombre de otros clientes. Los proxies brindan soporte para el almacenamiento en caché de mensajes HTTP y el filtrado de solicitudes HTTP, entre otros roles interesantes) en el medio.

Una **conexión** se define como un *circuito virtual* que está compuesto por agentes HTTP, incluido el navegador, el servidor y el intermediario proxys que participan en el intercambio.

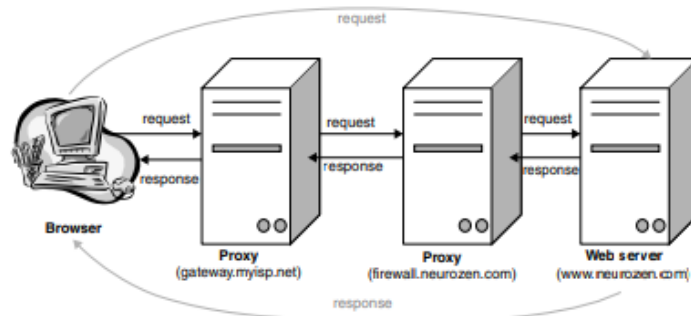


Ilustración 7. Circuito virtual de solicitud/respuesta.

Protocolo sin estado

Cuando un protocolo admite un 'estado', esto significa que proporciona la interacción entre el cliente y servidor para contener una secuencia de comandos. El servidor es necesario para mantener el 'estado' de la conexión a lo largo de la transmisión de comandos sucesivos, hasta que la conexión se termina.

La secuencia de comandos transmitidos y ejecutados a menudo se llama **sesión**. Muchos protocolos de Internet, incluidos FTP, SMTP y POP son protocolos 'con estado'.

Por el contrario, se dice que **HTTP es "sin estado"**, esto simplificó las cosas, pero también impuso limitaciones en las capacidades de las aplicaciones Web, ya que por definición la vida útil de una conexión era el intercambio de una sola solicitud/respuesta. Esto **significaba que**: no había manera de mantener información persistente sobre una 'sesión' de interacciones sucesivas entre un cliente y un servidor, no había forma de 'agrupar' las solicitudes (algo útil para pedirle por ejemplo a un servidor web una página HTML y todas las imágenes a las que hace referencia durante el transcurso de una conexión). De ahí, se derivaron **las cookies** (como un mecanismo de estado de aplicaciones Web).

Estructura de mensajes HTTP

```
METHOD /path-to-resource HTTP/version-number
Header-Name-1: value
Header-Name-2: value

[ optional request body ]
```

Ilustración 8. Estructura de mensaje HTTP

Cada solicitud comienza con la línea de solicitud especial, que contiene varios campos.

- El 'método' representa uno de varios métodos de solicitud admitidos, los principales entre ellos 'GET' y 'POST'.
- La '/ruta-al-recurso' representa la parte de la ruta de la URL solicitada.
- El 'número de versión' especifica la versión de HTTP utilizada por el cliente.
- Después de la primera línea, vemos una lista de encabezados HTTP, seguida de una línea en blanco, a menudo llamado <CR> (por 'retorno de carro y avance de línea'). La línea en blanco separa los encabezados de solicitud del cuerpo de la solicitud. Se sigue la línea en blanco (opcionalmente) por un cuerpo, que a su vez va seguido de otra línea en blanco que indica el final del mensaje de solicitud.

Un ejemplo de lo anterior:

```
GET /sj/index.html HTTP/1.1
Host: www.mywebsite.com
```

Ilustración 9. Ejemplo de una solicitud HTTP

Aquí, <http://www.mywebsite.com/sj/index.html> es la URL solicitada.

Por otro lado, a través de la siguiente URL, podrán identificar las partes la que componen:

scheme://host[:port#]/path.../[;url-params][?query-string][#anchor]

- **esquema:** esta parte de la URL designa el protocolo que se utilizará (p.e. http o ftp), precede a los 2 puntos y 2 diagonales.
- **host:** nombre de la dirección IP del servidor web al que se accede.
- **#puerto:** esta es una parte opcional de la URL que designa el # de puerto que escucha el servidor web de destino. (El puerto predeterminado para servidores HTTP es 80, si se utiliza alguno alternativo se debe especificar).
- **ruta:** lógicamente hablando esta es la ruta del sistema de archivos desde el directorio raíz del servidor al documento deseado.
- **url-params:** se usa con más frecuencia para identificadores de sesión en servidores web que soportan la API Java Servlet.
- **cadena de consulta:** es opcional de la URL contiene otros parámetros dinámicos asociados con la solicitud. ?var1=val1& . Por lo general se producen como el resultado de variables ingresadas por el usuario en un formulario HTML.
- **ancla:** es una referencia a un marcador de posición dentro del documento solicitado.

Ejercicio. De la siguiente URL proporcionada identifica los elementos solicitados:

<http://www.mywebsite.com/sj/test?id=8079?name=Patty&x=true#stuff>

Esquema: http

Host: www.mywebsite.com

Ruta: /sj/test

Url-params: id=8079

Cadena de consulta: name=Patty&x=true

Ancla: stuff

1.2.4. Evolución de Navegadores Web

Para comenzar a abordar este tema es necesario comprender la siguiente definición:

Navegador Web: es un programa informático que permite al usuario ingresar a las páginas Web que desee. Teniendo la dirección URL (Uniform Resource Locator) o a través de un buscador.

A continuación, te pido que realices la tarea: *Línea de tiempo de los navegadores Web*,
localizadas en classroom.

1.3 Estándares para la Web

[Ver presentación](#)

1.4 Entornos de desarrollo para aplicaciones Web

Un IDE (Integrated Development Environment) es un programa informático compuesto por un conjunto de herramientas de programación. Consiste en un editor de código, un compilador, un depurador (nos permite localizar y solucionar errores de una manera mucho más rápida y fácil) y un constructor de interfaz gráfica (GUI). Con una estructura definida, reusable en el que sus componentes facilitan la creación de aplicaciones web.

Puede dedicarse en exclusiva a un solo lenguaje de programación o bien puede utilizarse para varios.

Lo primero que debemos hacer a la hora de elegir un entorno de desarrollo o IDE (del inglés Integrated Development Environment) es **tener claro los lenguajes de programación** que vamos a utilizar. Como estamos hablando de desarrollo web tenemos

claro que si o si vamos a utilizar **HTML, CSS y JavaScript** (Frontend). En la parte de servidor (Backend) tenemos varias opciones, las más comunes son **PHP, Java y ASP.NET**. Dependiendo del Backend elegiremos un IDE u otro.

Cuando estamos comparando los IDEs que nos ofrece el mercado debemos plantearnos ciertas cuestiones:

- Coloreado de sintaxis para una mejor legibilidad.
- Que permita insertar trozos de código o snippets.
- Integración con un sistema de control de versiones.
- Poder crear proyectos a partir de plantillas o templates.
- Función de autocompletado de código.
- Ejecución en modo debug.
- Buscar y remplazar código.
- Refactorizar código.

Si nos basamos en el lenguaje de programación del lado del servidor podemos agrupar los IDE's en las siguientes categorías:

PHP

- [Netbeans](#)
- [Eclipse](#)
- [Aptana](#)
- [Visual Studio Code](#)
- [WebMatrix](#)
- [Atom](#)

Java

- [Netbeans](#)